Numerical Methods

- Certain special type of algebraic methods which give approximate solutions to scientific, engineering and mathematical problems by arithmetic operations on numbers.
- Large number of arithmetic calculations need to be performed that require fast and efficient computing devices.
- These methods are generally iterative in nature ie. One mathematical formula is repeatedly applied to get successive approximates to a solution.
- Three types of errors generally effect these methods: round off errors, truncation errors and errors due to finite representation of numbers like 1/3.
- Various types of numerical methods are: methods of finding roots of equation, numerical integration, numerical differentiation, Interpolation and regression analysis etc.

Numerical Analysis.

- Application of numerical methods to mathematical problems to get approximate results and then interpret and analyze the results along with the errors associated.
- Numerical Analysis consists of formulation and application of algorithms for numerical solution of problems and interpretation of outputs.
- Process of numerical analysis:
  - Formulation of mathematical model that express essential features of physical system.
  - Selection of appropriate numerical method keeping in view type of computing device available along with the accuracy and speed of execution desired.
  - Implementation of method to arrive at solution ie. Design of algorithm, writing program and implementing it.
  - Verification of result for desired level of accuracy, modify the model if required.

Numerical Methods vs. Numerical Analysis.

- Numerical method is a special type of method used to obtain approximate solution to a specific mathematical problem.
- Numerical Analysis is application of appropriate numerical method to a problem in a systematic way to get solution and make interpretation of the solution.

Errors

- Error is difference between exact and approximate values of certain result.
- Err=EV-AV
- It can be measured in three different ways:
  - Absolute way
  - Relative way
  - Percentage way
- Absolute Error: Absolute difference between EV and AV ie. ErrAbs = |EV-AV|
- Relative Error: Ratio of absolute error to exact or approximate value of result ie. ErrRel=|EV-AV|/EV or |EV-AV|/AV. Mostly AV is used as denominator.
- Percentage Error: ErrPer=ErrRel * 100. Percentage Error reveals better meaningful results.

Numerical:

- Two numbers are given 15.2 and 0.0893. Find sum of two numbers in exact manner. Find sum upto three significant digits. Determine absolute, relative and percentage errors.

Types of errors

- Broadly divided into four types:
  - Inherent Errors
  - Numerical Errors
  - Modelling Errors
  - Blunders.
- Inherent Errors: Errors that are inherent with the data collected and supplied due to problem in instrument or with investigator. Two types of inherent errors:
  - Data Errors: Empirical errors due to inexactness of instruments and experimental means used.
  - Conversion Errors: Representation errors due to inherent limitations of computing machinery. For eg. 0.1 cannot be represented in memory accurately.

Types of errors contd..

- Numerical Errors: Procedural errors that take place during the process of application of numerical methods. Two types of numerical errors:
  - Round off Errors: When a calculated figure is rounded off to a fixed number of digits, error is introduced. With symmetric rounding off, cumulative effect of such errors become less significant. With asymmetric rounding off, cumulative effect becomes more significant. Round off errors are committed in two ways:
    - Chopping: Asymmetric rounding off
    - Symmetric Rounding:
  - Truncation Errors: When certain infinite processes are replaced by finite ones, truncation error is introduced.
    - For eg. $e^x=1+x+x^2/2!$..... Is replaced by $e^x=1+x$ upto 2 terms only.

Types of errors contd..

- Modeling Error:
  - Error that creeps in to the output on evaluation due to certain simplifying assumptions in the formulation of mathematical models.
- Blunder:
  - Error that takes place purely due to limitations and Imperfections of human resources. Certain causes:
    - Inability to understand the problem.
    - Drawing wrong assumptions.
    - Selection of wrong algorithm.
    - Selection of wrong numerical method etc.

Some basic Concepts

- An approximate number is defined as the number that is used as an approximation to an exact number and differs slightly from exact number. For eg. ∏.
- Significant Digits: Any digit is significant digit except when it is used to fix the decimal point. Best way is to write number in scientific notation. For eg. 0.00263 when written in scientific notation $.263X10^{-2}$ There are 3 significant digits.
- Complements: Positive numbers are stored as it is in memory. To store negative numbers complements are used.

Some basic concepts

- 1’s complement of a binary number is obtained by inverting the bits.
- For eg. 44=binary 101100. Its 1’s complement is 010011. So 010011 is the binary representation of -44 in 1’s complement scheme.
- 2’s complement of a binary number is obtained by first taking 1’s complement and then adding 1 to the least significant bit. Alternatively examine the binary number from right to left, keep first non zero bit as it is and invert the subsequent bits.
- For eg. 44=binary 101100, 1’s complement is 010011 and 2’s complement is 010100. So 010100 is the binary representation of -44 in 2’s complement scheme.

Some basic concepts

- Storage of Integer numbers:
  - Signed representation: MSB holds sign and other bits hold magnitude of the number. For eg. 44 and -44 in 1 byte format are represented as 00101100 and 10101100 respectively.
  - 1's complement representation: Positive integers are stored as in signed representation. For negative integers, MSB holds 1 and other bits hold 1's complement of the integer. For eg. 44 and -44 in 1 byte format is represented as 00101100 and 11010011 respectively.
  - 2's complement representation: Positive integers are stored as in signed representation. For negative integers, MSB holds 1 and other bits hold 2's complement of the integer. For eg. 44 and -44 in 1 byte format is represented as 00101100 and 11010100 respectively.
- In all the representations, MSB is used to store the sign of integer number. If sign bit is 0, then it is positive number otherwise negative number is there.

Storage of numbers

- Storage of Real Numbers: Real numbers can be written in two formats:
  - Fixed format (without exponent)
  - Floating point format (with exponent): Here real number consists of two parts - mantissa and exponent. For eg. Real number $1.257x10^5$ in the scientific form will be written as 1.257E5 in floating point format. Decimal point can move freely from left to right but standard practice is to put it before the first non zero digit.
- Shifting of decimal point to left of the most significant digit is called normalization. Real numbers represented in this format are known as normalized floating point number. Normalization is used to preserve maximum number of useful digits.
- Suppose each word in memory can store 6 digits, 4 digits before decimal point and 2 digits after decimal point. Without normalization numbers between 0000.01 and 9999.99 can only be stored. After normalization numbers between $.1000*10^0$ and $.9999*10^{99}$ can be stored.

Storage of numbers contd..

- In normalized floating point representation, Mantissa should satisfy the condition:
- For positive numbers 0.1<=mantissa<1.0
- For negative numbers -1.0<mantissa<=-0.1
- ie. 0.1<=|mantissa|<1.0
- For eg. 0.00125e-2 is expressed as 0.125e-4
- Floating point numbers are represented in memory in four consecutive bytes.



Storage of numbers contd..

- Exponent of floating point number can be positive or negative.
- One way is to reserve MSB of exponent for sign, then the exponent ranges from -128 to +127.
- Biased exponent can be used where constant 128 is added to the exponent so that it varies from 0 to 255.
- Storing real number in 32 bit format:
  - Convert the number into binary.
  - Normalize it
  - Extend the matissa to 23 bits by adding 0s towards right side.
  - Take 2's complement if the number is negative.
  - Add sign bit
  - Add 128 to the exponent and convert it into binary.
  - Store all the bits.
  - Eg. 13.5 (10000100001101100000000000000000) and -13.5 (10000100100101000000000000000000)

Storage of numbers contd..

- Possible range of values:
  - Largest floating point number will have 0 as sign bit followed by all 1s for mantissa and 127 as actual exponent. Order of magnitude in decimal can be found out as $2^{127}=10^{x}$ which comes out to be x=38.
  - Smallest floating point number will have the actual exponent -128. It will be roughly of the order of 10^{-38}
  - Range of floating point numbers is $10^{\pm 38}$

Floating Point Arithmetic

- Addition: Exponents of two numbers must be made equal by increasing the smaller exponent and add mantissas.
- Subtraction: Same as addition ie adding a negative number.
- Multiplication: Multiply mantissas and add exponents.
- Division: Mantissa of numerator is divided by mantissa of denominator. Denominator exponent is subtracted from numerator exponent.

Floating Point Arithmetic

- Consequences of Normalized Floating Point Representation of Numbers
  - Non Associativity of Arithmetic
    - As numbers need to be truncated to fit into the space. Surprising results are produced sometimes. For eg. 2/3*6 =4. .6667 added 6 times produce .4001E1(using round off) or .3997e1 (using truncate)
    - Associative and distributive laws or arithmetic are not always valid ie. (a+b)-c ≠ (a-c)+b and a(b-c) ≠ (ab-ac)
      - For eg. a=.5665E1, b=.5556E-1,c=.5644E1. Evaluating it we get (a+b)-c ≠ (a-c)+b.
      - For eg. a=.5555E1, b=.4545E1, c=.4535E1. Evaluating it we get a(b-c) ≠ ab-ac
    - Disparity in results is there because difference of two almost equal numbers is involved. So, subtraction operation should be eliminated altogether by following alternate ways.

Floating Point Arithmetic

- For eg. Instead of evaluating (1-cos x) its better to evaluate $2\sin^2 x/2$ if value of cos x is close to 1.

- Concept of Zero in floating point number:
  - Zero has a precise meaning in arithmetic, but in normalized floating point mode, exact equality of a number to zero can never be assured.
  - For eg. Roots of quadratic equation $x^2+2x-2=0$ is $-1\pm\sqrt{3}$ which in floating point representation is .7320E0 and -.2732E1. When we put .7320E0 in the equation we get -.1000E-2 ie. Not exactly zero.

Floating Point Arithmetic

- pitfalls in computing:
  - Suppose ax+by=c and px+qy=r are to be solved for a=2.500, b=5.200, c=6.200, p=1.251, q=2.605, r=3.152.
    - Using ordinary arithmetic, solution is x=-.32794E2, y=.16958E2. Using floating point arithmetic, we get x=-.3217E2, y=.1666E2. Solutions agree in only 2 most significant digits. This is because the method involves difference of almost equal numbers.
    - If q is changed to 2.606, using floating point arithmetic we get x=-.2352E2, y=.1250E2. Such problem is said to be **illconditioned.**

Floating Point Arithmetic

- Suppose for equations ax+by=c, px+qy=r, we have a=.2E-4, b=2,c=2,p=1,q=2,r=4. Using floating point arithmetic we get x=0, y=1. If we interchange equations, we get x=2, y=1 which is nearer to true solution x=2.00004, y=.99998. Simply interchanging equations brings improvement.

Precautions that can be taken

- Minimize number of arithmetic operations. Compute n*x instead of x+x+x+x upto n terms.
- Avoid subtraction of two nearly equal numbers.
- Do not test a floating point number for zero.
- Rebuild your formula if possible so that you use original data rather than derived data.
- Reconstruct your formula to reduce arithmetic operations.
- Avoid multiplication of large numbers to avoid overflow.
- Use integer arithmetic if possible.
- After deciding numerical algorithm, always check if it leads to numerical instability or not.
- If serious rounding off errors and loss of significant digits are involved use double precision mode for floating point numbers.

Error Propagation

Suppose x_a is approximation to x and y_a is approximation to y.

$e_{ax}=x-x_a$, $e_{ay}=y-y_a$

$e_{rx}=(x-x_a)/x$, $e_{ry}=(y-y_a)/y$

- Error Propagation in Addition operation
  - $|(x+y)-(x_a+y_a)|=|e_{ax}+e_{ay}|$
  - $|e_{ax}+e_{ay}|<=|e_{ax}|+|e_{ay}|$ (By triangle inequality)
- Error Propagation in Subtraction operation
  - $|(x-y)-(x_a-y_a)|=|e_{ax}-e_{ay}|$
  - $|e_{ax}-e_{ay}|<=|e_{ax}|+|e_{ay}|$ (By triangle inequality)

Error Propagation

- For multiplication and division, its convenient to take relative error.
  - $(x-x_a)/x=e_{rx}$
  - $x-x_a=xe_{rx}$
  - $x_a=x-xe_{rx}$
  - $x_a=x(1-e_{rx})$
  - As relative error can be negative we take $x_a=x(1+e_{rx})$, $y_a=y(1+e_{ry})$
- Error Propagation in Multiplication operation
  - $x_a y_a=xy(1+e_r)$ where $e_r=e_{rx}+e_{ry}$
- Error Propagation in Division operation
  - $x_a/y_a=x/y(1+e_r)$ where $e_r=e_{rx}-e_{ry}$

Non-Linear Equations

- Most of the non linear equations can be solved algebraically (algebraic or analytical solution). Sometimes algebraic solution exist but is extremely complicated and impractical, for that numerical solution is found.
- Roots of an equation f(x)=0 is also called zero(s) of equation.
- Types of Non-Linear Equations:
  - Polynomial: For eg. $x^2+2x+3=0$
    - nth degree polynomial has n roots which can be
      - Real and different
      - Real and repeated
      - Complex
  - Transcendental (Non-Polynomial): For eg. $xe^x-xsinx=0$

Non-Linear Equations

- Two methods to obtain solutions of non-linear equations:
  - Direct Method:
    - Roots are obtained in finite number of steps
    - All roots are found at the same time.
  - Iterative Method (Also called Trial & Error Method):
    - Based on the idea of successive approximations.
    - Start with one or more initial approximation to the root.
    - Generally gives one root at a time.
  - Iterative methods are best suited for computers as:
    - They can be concisely expressed as computational algorithms.
    - Possible to formulate algorithms that can handle class of similar problems.
    - Round-off errors are negligible in iterative methods as compared to direct methods.

Non-Linear Equations

- Choosing Initial Approximation:
  - Plot the function or tabulate it. Roots of the equation are the points where curve intersects x-axis. For eg. 2^x-x-3=0 has roots between (-3,-2) and (2,3)
  - If f(x) is a continuous function and f(a)*f(b)<0 then the root lies in the interval (a,b) or (b,a) if b<a
- When to terminate iterative procedure:
  - When the required degree of accuracy is achieved. Suppose prescribed tolerance is epsilon
  - Termination Criteria 1: $|f(x_{i+1})|<=epsilon$
  - Termination Criteria 2: $|x_{i+1}-x_i|<=epsilon$
  - Termination Criteria 3: $|(x_{i+1}-x_i)/x_{i+1}|<=epsilon$ for $x_{i+1} \neq 0$

Non-Linear Equations

- Bisection Method:
  - Two initial approximations x_1, x_2. x_3 is taken as midpoint of x_1, x_2.
  - If $f(x_3) = 0$, x_3 is the root.
  - If $f(x_1)*f(x_3)<0$ then root lies in (x_1,x_3)
  - If $f(x_1)*f(x_3)>0$ then root lies in (x_3,x_2)
  - Root is enclosed in halved search interval with each iteration.
  - Iterative cycle is terminated when interval becomes smaller than prescribed tolerance.

Bisection Method

- If the method is repeated n times, search interval is reduced to $(x_2-x_1)/2^n$ and the method is stopped when $(x_2-x_1)/2^n <= \text{epsilon}$.
- Formula can be reduced to

  $n >= (\log(x_2-x_1) - \log \text{epsilon})/\log 2$
- Eg. If root lies between (1,2) then for epsilon 10^{-2}, 7 iterations are required.
- Bisection method requires large number of iterations to achieve reasonable degree of accuracy.
- Problem: Given that one root of $x^3-4x-9=0$ lies between 2.625 and 2.75. Find root correct to four significant digits.

Bisection Method

- Algorithm

```
Begin
    read: x1,x2
    read: epsilon
    do
        set x3=(x1+x2)/2
        If f(x3)=0, then break
        if(f(x1)*f(x3))<0), then:
            set x2=x3
        else
            set x1=x3
        end if
    while(|x1-x2|>epsilon)
    write: x3, ‘as the approx. root’
End
```

False Position Method

- Bisection method guarantees convergence but the convergence is quite slow.
- False position method is also called regula-falsi method or method of linear interpolation.
- We find initial interval (x_1,x_2), find x_3 based on the formula $x_3=(x_1f(x_2)-x_2f(x_1))/(f(x_2)-f(x_1))$ and change the interval according to the sign of $f(x_1)$ and $f(x_3)$ ie if $f(x_1)*f(x_3)<0$ then new interval is (x_1,x_3) otherwise (x_3,x_2)
- Formula for x_3 is computed based on the concept of slope of the straight line joining two points representing initial approximation interval.
- Problem: One root of non-linear equation $x^3-2x-5=0$ lies between (1.75,2.5), Find root correct to four significant digits.

False Position Method

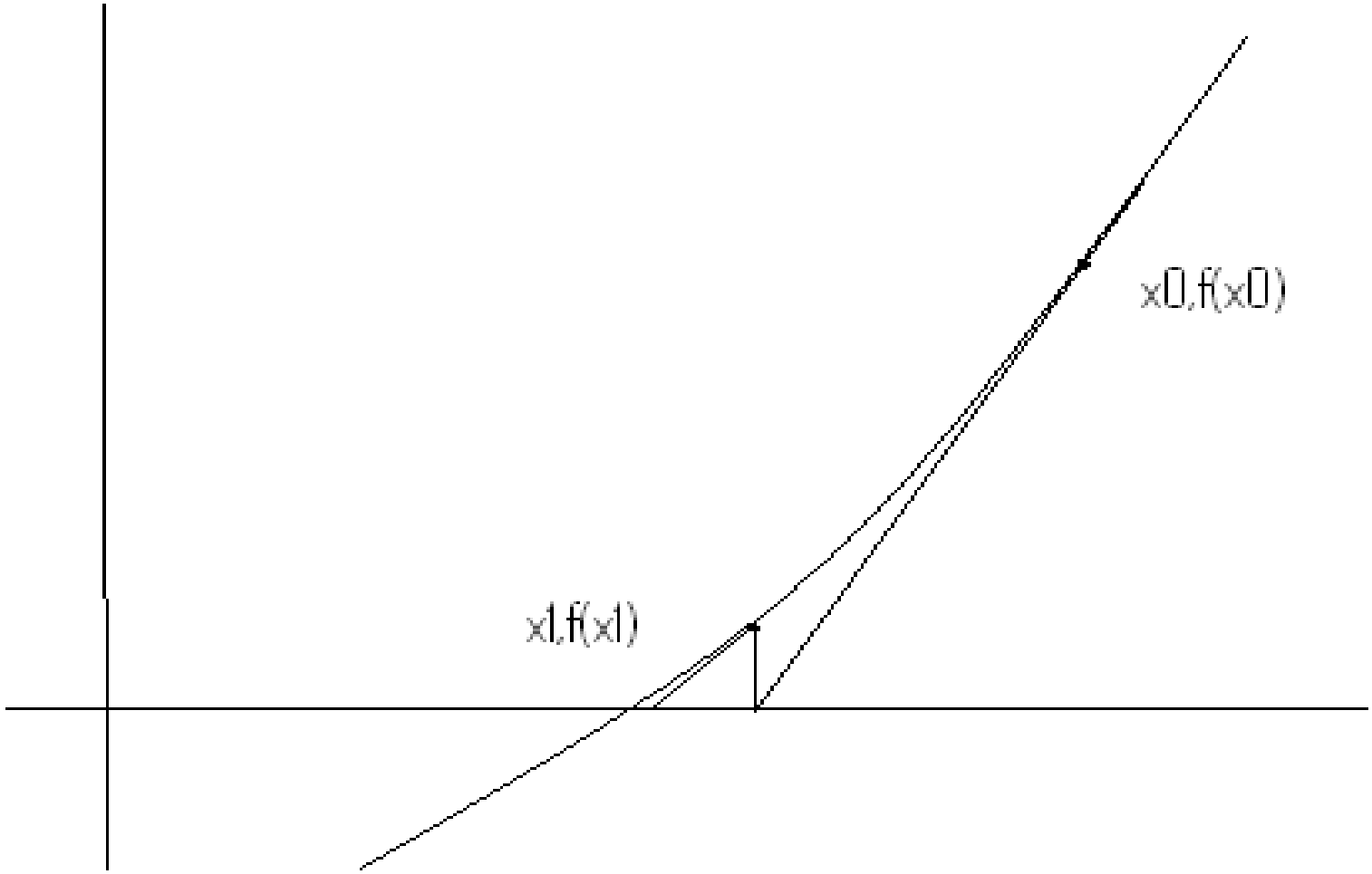
- Algorithm

```
Begin
    read: x1,x2,epsilon,delta
    Set f1=f(x1),f2=f(x2)
    If (f1*f2>0) then
       write: “Initial approximation is unsuitable” and Exit
    End If
    do
        If (|f2-f1|<=delta) then
           write: “Slope of the function becomes too small” and Exit
        End If
        set x3=(x1*f2-x2*f1)/(f2-f1)
        Set f3=f(x3)
        if(f1*f3)<0), then:
              set x2=x3,f2=f3
        else
              set x1=x3,f1=f3
        end if
    while(|x1-x2|>epsilon)
    write: x3, ‘as the approx. root’
End
```

Newton-Raphson Method

- Also called Newton's method of tangents.
- One of the fastest iterative methods.
- One initial approximation is required.
- Method is very sensitive to initial approximation that's why we keep a check on number of iterations.
- Analytical Derivation:
  - $f(x)=0$
  - $f(x0+h)=0$ where x0 is initial approximation, h is correction.
  - Expanding by Taylor series and ignoring higher order terms we get $f(x0)+hf^1(x0)=0$
  - $x1=x0-f(x0)/f^1(x0)$
- Geometric Derivation (diagram is on next slide):
  - $\tan\Theta = f(x0)/(x0-x1)=f^1(x0)$
  - Solving it we get, $x2=x1-f(x1)/f1(x1)$
- If the slope is too small it will complicate computations, so limit is put on the lower bound of slope.
- **Problem**: One root of non linear equation $x^3-4x-9=0$ lies between 2.625 and 3.0 Find root correct to four significant digits.

Newton-Raphson Method



- Algorithm:

Begin

read: x_0, epsilon, delta, n

for i=1 to n by 1 do

if (|f'(x_0)|<=delta) then

write: "Slope of f(x) becomes too small near x = ", x_0

exit

endif

set x_1=x_0-f(x_0)/f'(x_0)

set rel_error=|((x_1-x_0)/x_1)|

set x_0=x_1

if(rel_error<=epsilon) then

write: x_1, "as the approximate root"

exit

endif

end for

write: "Solution does not converge in ",n, "iterations".

End

Reference(s)

- Salaria, R. S. "Numerical methods: A computer oriented approach." (1996).
- Rajaraman, Vaidyeswaran. *Computer oriented numerical methods*. PHI Learning Pvt. Ltd., 1993.